

Scheduling with systemd on Linux

28-02-2026

Alright team  — this is the session where you level up from “I run Linux” to:

”I control what runs on Linux... and when it runs.”

We’re diving into:

- systemd services
- Basic unit files
- systemd timers (modern scheduling)

By the end, you’ll:

- Understand how your system boots and manages services
- Create your own service units
- Schedule tasks like a pro (without cron)
- Override and customize services safely

1 What is systemd? (And Why You Should Care)

Before scheduling anything, you need to understand what’s running your system.

On modern Debian-based systems:

- **Debian**
- **Ubuntu**
- **Linux Mint**

The init system is:

systemd

An *init system* is the first userspace process started by the kernel.

Check it:

```
$ ps -p 1 -o comm=
```

You'll see:

```
systemd
```

What Does systemd Do?

- Starts services during boot
- Manages background processes (daemons)
- Handles logging (via journald)
- Mounts filesystems
- Manages network services
- Handles timers (our main topic today)

If Linux is a city 
systemd is the mayor.

Helpful Reading

- [geeksforgeeks-linux-systemd-and-its-components](#)
- [geeksforgeeks-introduction-to-systemd](#)

2 Understanding Units in systemd

Everything in systemd is a **Unit**.

A unit is a configuration file that tells systemd what to manage.

Common Unit Types

Type	Extension	Purpose
Service	.service	Background services
Timer	.timer	Scheduling
Mount	.mount	Mount points
Target	.target	Group of units
Socket	.socket	Socket activation

Example:

```
apache2.service  
ssh.service  
cron.service
```

To list all units:

```
$ systemctl list-units
```

To list all installed units:

```
$ systemctl list-unit-files
```

3 Installing Apache (Service Example)

Let's install something real:

```
$ sudo apt update  
$ sudo apt install apache2
```

This installs:

```
apache2.service
```

Now check its status:

```
$ systemctl status apache2
```

You'll see:

- Loaded
- Active
- Main PID
- Logs

4 Managing Services with systemctl

This is daily Linux admin stuff.

Start a Service

```
$ sudo systemctl start apache2
```

Stop a Service

```
$ sudo systemctl stop apache2
```

Restart

```
$ sudo systemctl restart apache2
```

Reload vs Restart (Important!)

- `reload`
 - Reloads configuration
 - Does NOT stop process
 - Keeps connections alive
- `restart`
 - Stops service
 - Starts again
 - Drops active connections

Example:

```
$ sudo systemctl reload apache2
```

Enable at Boot

```
$ sudo systemctl enable apache2
```

This creates a symlink in:

```
/etc/systemd/system/
```

Disable at Boot

```
$ sudo systemctl disable apache2
```

Mask a Service

Masking = completely block it from starting.

```
$ sudo systemctl mask apache2
```

Unmask:

```
$ sudo systemctl unmask apache2
```

→ More explanation: [geeksforgeeks-mask-a-systemd-unit-in-linux](https://www.geeksforgeeks.org/mask-a-systemd-unit-in-linux/)

5 Where Are Unit Files Stored?

Systemd has priority layers.

Main Directories

Directory	Purpose
/usr/lib/systemd/system/	Default package units
/lib/systemd/system/	(Debian equivalent)
/etc/systemd/system/	Custom & overrides

Priority Order

Highest priority:

```
/etc/systemd/system/
```

So if you edit there — it overrides the default.

6 Anatomy of a Service File

Let's inspect:

```
$ systemctl cat apache2.service
```

Typical structure:

```
[Unit]
Description=...
After=network.target

[Service]
Type=forking
ExecStart=/usr/sbin/apachectl start
ExecReload=/usr/sbin/apachectl graceful
ExecStop=/usr/sbin/apachectl stop

[Install]
WantedBy=multi-user.target
```

[Unit] Section

Defines metadata and dependencies.

Common options:

- Description
- After=
- Requires=

⚙️ [Service] Section

How the service runs.

Important fields:

Option	Meaning
ExecStart	Command to start
ExecStop	Command to stop
Restart	Restart policy
User	Run as user
Type	simple, forking, oneshot

🔌 [Install] Section

Defines how it integrates into boot.

```
WantedBy=multi-user.target
```

Means: Start in normal multi-user mode.

7 </> Creating Your Own Service

Let's create one.

🎯 Goal: Run a simple script

Create script:

```
$ sudo nano /usr/local/bin/hello.sh
```

Add:

```
#!/bin/bash
echo "Hello from systemd!" >> /tmp/hello.log
```

Make executable:

```
$ sudo chmod +x /usr/local/bin/hello.sh
```

Create Service File

```
$ sudo nano /etc/systemd/system/hello.service
```

Add:

```
[Unit]
Description=My Hello Service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/hello.sh

[Install]
WantedBy=multi-user.target
```

Reload systemd:

```
$ sudo systemctl daemon-reload
```

 This is IMPORTANT anytime you:

- Create
- Edit
- Delete

unit files.

Start service:

```
$ sudo systemctl start hello.service
```

Check result:

```
$ cat /tmp/hello.log
```

Boom ⚡
You just created a service.

8 ✎ Editing & Overriding Units (The Smart Way)

Never edit files in `/lib/systemd/system/`.

Instead:

```
$ sudo systemctl edit apache2
```

This creates:

```
/etc/systemd/system/apache2.service.d/override.conf
```

Add:

```
[Service]  
Restart=always
```

Save.

Reload daemon:

```
$ sudo systemctl daemon-reload
```

Restart service.

9 🕒 systemd Timers (Modern Scheduling)

Now we level up.

Timers = replacement for cron.

Why Use systemd Timers Instead of Cron?

- Integrated with services

- Better logging
- More control
- Dependency aware
- Can use calendar expressions

i Note that systemd is more complex than Cron and requires systemd to work on your system.

🕒 Timer Structure

A timer requires:

1. A `.service` file
2. A `.timer` file

10 📧 Example: Send Message to All Users

We'll simulate the example from the referenced video.

Create Script

```
$ sudo nano /usr/local/bin/wall-msg.sh
```

Add:

```
#!/bin/bash  
wall "System reminder: Stay awesome!"
```

Make executable:

```
$ sudo chmod +x /usr/local/bin/wall-msg.sh
```

Create Service File

```
$ sudo nano /etc/systemd/system/wall-msg.service
```

```
[Unit]
Description=Wall Message Service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/wall-msg.sh
```

Create Timer File

```
$ sudo nano /etc/systemd/system/wall-msg.timer
```

```
[Unit]
Description=Run wall message every minute

[Timer]
OnCalendar=*-*-* *:*:00
Persistent=true

[Install]
WantedBy=timers.target
```

Reload daemon:

```
$ sudo systemctl daemon-reload
```

Enable timer:

```
$ sudo systemctl enable wall-msg.timer
```

Start timer:

```
$ sudo systemctl start wall-msg.timer
```

Check timers:

```
$ systemctl list-timers
```

🔊 You are now scheduling tasks.

📅 Understanding OnCalendar

Examples:

Expression	Meaning
daily	Once per day
weekly	Weekly
Mon *--* 09:00:00	Mondays 9AM
*--01 00:00:00	First day of month

Test calendar expression:

```
$ systemd-analyze calendar "Mon *--* 09:00:00"
```

⚙️ Other Timer Options

Option	Meaning
OnBootSec=	After boot
OnUnitActiveSec=	After last run
Persistent=true	Run missed jobs after reboot

11 ⚖️ Timer vs Cron Comparison

Feature	Cron	systemd Timer
Logging	Basic	journalctl
Dependencies	No	Yes
Missed Runs	No	Yes (Persistent)
Integration	Separate	Native

12 🛠 Debugging Services & Timers

Logs:

```
$ journalctl -u wall-msg.service
```

Live logs:

```
$ journalctl -f
```

Check failures:

```
$ systemctl --failed
```

13 🌐 Real World Use Cases

- Backup scripts
- Log cleanup
- Health checks
- Auto-updates
- Dev server restarts
- IoT tasks

14 ⚠ Common Mistakes

- Forgetting `daemon-reload`
 - Editing `/lib/systemd/system/` directly
 - Wrong file permissions
 - Forgetting `WantedBy=`
-

Final Mental Model

- Service = What to run
- Timer = When to run
- Unit file = Configuration blueprint
- systemctl = Control center

You Now Know

- How systemd works
- How to control services
- How to create units
- How to override safely
- How to schedule with timers
- How to debug issues

You are no longer a Linux user.
You're becoming a Linux operator. 